

LAPORAN PRAKTIKUM
SISTEM OPERASI

MODUL

7



Oleh:

(Rifqi M Ramdani)

(2311104044)

PROGRAM STUDI S1 REKAYASA LUNAK
DIREKTORAT KAMPUS PURWOKERTO
UNIVERSITAS TELKOM

2025

BAB I

PENDAHULUAN

A. Dasar Teori

Semaphore merupakan mekanisme sinkronisasi yang digunakan dalam sistem operasi untuk mengatur akses terhadap sumber daya bersama dalam lingkungan komputasi paralel atau konkuren. Secara teknis, semaphore bekerja sebagai variabel penanda yang mengontrol urutan eksekusi antar proses melalui operasi `sem_wait` untuk menahan proses dan `sem_post` untuk melepaskan atau memberikan sinyal kepada proses lain. Konsep ini sangat krusial dalam menyelesaikan permasalahan *race condition* dan diimplementasikan secara luas pada model *Producer-Consumer*, di mana dua proses atau lebih harus berkoordinasi agar pengambilan dan pembuatan data berjalan secara selaras tanpa adanya data yang hilang atau tumpang tindih.

B. Tujuan

Tujuan utama dari praktikum ini adalah agar mahasiswa memiliki kemampuan dalam mengimplementasikan konsep semaphore secara benar di dalam sebuah program. Melalui pengerjaan jurnal ini, mahasiswa diharapkan mampu mengelola proses-proses yang berjalan secara konkuren di sistem operasi Linux, serta memahami cara melakukan sinkronisasi urutan output pada beberapa proses yang berbeda. Selain itu, praktikum ini bertujuan untuk melatih kemandirian mahasiswa dalam memecahkan masalah logika pemrograman sistem menggunakan alat bantu seperti compiler GCC di lingkungan VirtualBox.

BAB II je.

HASIL

A. UNGUIDED (Tugas Mandiri)

- 1. [50 poin] Buatlah 3 buah proses yaitu P1, P2 dan P3. P1 selalu menampilkan “kwak”, P2 selalu menampilkan “kwik”, P3 selalu menampilkan “kwek”. Menggunakan 3 proses tersebut dan beberapa buah semaphore, buatlah program yang dapat menampilkan: Kwak Kwik Kwek Kwak Kwik Kwek ...**

Jawab

Source Code

Activities Terminal Tue 19 May, 10:45 xinu@xinu-develop-end: ~

File Edit View Search Terminal Help

GNU nano 2.2.6 File: soall.c

```
/* Nama : Rifqi M Ramdani */
/* NIM : 2311104044 */

#include <stdio.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>

sem_t sem1, sem2, sem3;

void* p1(void* arg) {
    while(1) {
        sem_wait(&sem1);
        printf("Kwak\n");
        sleep(1);
        sem_post(&sem2);
    }
}

void* p2(void* arg) {
    while(1) {
        sem_wait(&sem2);
        printf("Kwik\n");
    }
}
```

[Read 54 lines]

Get Help WriteOut Read File Prev Page Cut Text Cur Pos
Exit Justify Where Is Next Page UnCut Text To Spell

Activities Terminal Tue 19 May, 10:45 xinu@xinu-develop-end: ~

File Edit View Search Terminal Help

GNU nano 2.2.6 File: soall.c

```
int main() {
    pthread_t t1, t2, t3;

    sem_init(&sem1, 0, 1);
    sem_init(&sem2, 0, 0);
    sem_init(&sem3, 0, 0);

    pthread_create(&t1, NULL, p1, NULL);
    pthread_create(&t2, NULL, p2, NULL);
    pthread_create(&t3, NULL, p3, NULL);

    pthread_join(t1, NULL);
    pthread_join(t2, NULL);
    pthread_join(t3, NULL);

    return 0;
}
```

Get Help WriteOut Read File Prev Page Cut Text Cur Pos
Exit Justify Where Is Next Page UnCut Text To Spell

Output

[illegible]

2. [50 poin] Buatlah proses bernama produser yang memproduksi bilangan 1-1000. Buatlah proses bernama konsumen yang akan menampilkan nilai yang diproduksi oleh produser. Gunakan semaphore! Produser memproduksi nilai 1 Konsumer menampilkan nilai 1 Produser memproduksi nilai 2 Konsumer menampilkan nilai 2 Produser memproduksi nilai 1000 Konsumer menampilkan nilai 1000

Jawab

Source code

```
Activities Terminal Tue 19 May, 11:11
xinu@xinu-develop-end: ~

File Edit View Search Terminal Help
GNU nano 2.2.6 File: soal2.c

/* Nama : Rifqi M Ramdani */
/* NIM : 2311104044 */

#include <stdio.h>
#include <pthread.h>
#include <semaphore.h>

sem_t sem_prod, sem_cons;
int data;

void* produser(void* arg) {
    int i;
    for(i = 1; i <= 1000; i++) {
        sem_wait(&sem_prod);
        data = i;
        printf("Produser memproduksi nilai %d\n", data);
        sem_post(&sem_cons);
    }
    return NULL;
}

void* konsumen(void* arg) {
    int i;
    for(i = 1; i <= 1000; i++) {
        sem_wait(&sem_cons);
        printf("Konsumer menampilkan nilai %d\n", data);
        sem_post(&sem_prod);
    }
    return NULL;
}

[ Read 49 lines ]
Get Help WriteOut Read File Prev Page Cut Text Cur Pos
Exit Justify Where Is Next Page UnCut Text To Spell
```

```
Activities Terminal Tue 19 May, 11:11
xinu@xinu-develop-end: ~

File Edit View Search Terminal Help
GNU nano 2.2.6 File: soal2.c

        sem_post(&sem_cons);
    }
    return NULL;
}

void* konsumen(void* arg) {
    int i;
    for(i = 1; i <= 1000; i++) {
        sem_wait(&sem_cons);
        printf("Konsumer menampilkan nilai %d\n", data);
        sem_post(&sem_prod);
    }
    return NULL;
}

int main() {
    pthread_t t_prod, t_cons;

    sem_init(&sem_prod, 0, 1);
    sem_init(&sem_cons, 0, 0);

    pthread_create(&t_prod, NULL, produser, NULL);
    pthread_create(&t_cons, NULL, konsumen, NULL);

    pthread_join(t_prod, NULL);
    pthread_join(t_cons, NULL);

    sem_destroy(&sem_prod);
    sem_destroy(&sem_cons);

    return 0;
}

Get Help WriteOut Read File Prev Page Cut Text Cur Pos
Exit Justify Where Is Next Page UnCut Text To Spell
```

Output

```
Activities Terminal Tue 19 May, 11:10
xinu@xinu-develop-end: ~

File Edit View Search Terminal Help
Konsumer menampilkan nilai 983
Produser memproduksi nilai 984
Konsumer menampilkan nilai 984
Produser memproduksi nilai 985
Konsumer menampilkan nilai 985
Produser memproduksi nilai 986
Konsumer menampilkan nilai 986
Produser memproduksi nilai 987
Konsumer menampilkan nilai 987
Produser memproduksi nilai 988
Konsumer menampilkan nilai 988
Produser memproduksi nilai 989
Konsumer menampilkan nilai 989
Produser memproduksi nilai 990
Konsumer menampilkan nilai 990
Produser memproduksi nilai 991
Konsumer menampilkan nilai 991
Produser memproduksi nilai 992
Konsumer menampilkan nilai 992
Produser memproduksi nilai 993
Konsumer menampilkan nilai 993
Produser memproduksi nilai 994
Konsumer menampilkan nilai 994
Produser memproduksi nilai 995
Konsumer menampilkan nilai 995
Produser memproduksi nilai 996
Konsumer menampilkan nilai 996
Produser memproduksi nilai 997
Konsumer menampilkan nilai 997
Produser memproduksi nilai 998
Konsumer menampilkan nilai 998
Produser memproduksi nilai 999
Konsumer menampilkan nilai 999
Produser memproduksi nilai 1000
Konsumer menampilkan nilai 1000
root@xinu-develop-end:/home/xinu#
```

```
Activities Terminal Tue 19 May, 11:10
xinu@xinu-develop-end: ~

File Edit View Search Terminal Help
root@xinu-develop-end:/home/xinu# nano soal2.c
root@xinu-develop-end:/home/xinu# gcc soal2.c -o soal2 -lpthread
root@xinu-develop-end:/home/xinu# ./soal2
Produser memproduksi nilai 1
Konsumer menampilkan nilai 1
Produser memproduksi nilai 2
Konsumer menampilkan nilai 2
Produser memproduksi nilai 3
Konsumer menampilkan nilai 3
Produser memproduksi nilai 4
Konsumer menampilkan nilai 4
Produser memproduksi nilai 5
Konsumer menampilkan nilai 5
Produser memproduksi nilai 6
Konsumer menampilkan nilai 6
Produser memproduksi nilai 7
Konsumer menampilkan nilai 7
Produser memproduksi nilai 8
Konsumer menampilkan nilai 8
Produser memproduksi nilai 9
Konsumer menampilkan nilai 9
Produser memproduksi nilai 10
Konsumer menampilkan nilai 10
Produser memproduksi nilai 11
Konsumer menampilkan nilai 11
Produser memproduksi nilai 12
Konsumer menampilkan nilai 12
Produser memproduksi nilai 13
Konsumer menampilkan nilai 13
Produser memproduksi nilai 14
Konsumer menampilkan nilai 14
Produser memproduksi nilai 15
Konsumer menampilkan nilai 15
Produser memproduksi nilai 16
Konsumer menampilkan nilai 16
Produser memproduksi nilai 17
```


BAB III

PENUTUP

C. Kesimpulan

Berdasarkan hasil eksekusi program pada Modul 7, dapat disimpulkan bahwa penggunaan semaphore berhasil menjamin urutan eksekusi proses yang sinkron, terbukti dari output "Kwak", "Kwik", dan "Kwek" yang selalu muncul secara berurutan sesuai logika yang diinginkan. Pada implementasi soal kedua, mekanisme semaphore terbukti efektif dalam mengatur interaksi antara proses produser dan konsumen sehingga data bilangan 1 hingga 1000 dapat diproduksi dan ditampilkan secara bergantian tanpa terjadi kekacauan urutan. Secara keseluruhan, pemahaman mengenai manajemen proses konkuren dan penggunaan library pthread serta semaphore sangat penting bagi pengembang sistem untuk menciptakan aplikasi yang stabil dan efisien.